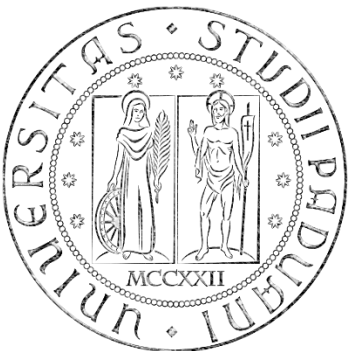


Biomedical Wearable Technologies for Healthcare and Wellbeing

HTTP

A.Y. 2024-2025
Giacomo Cappon



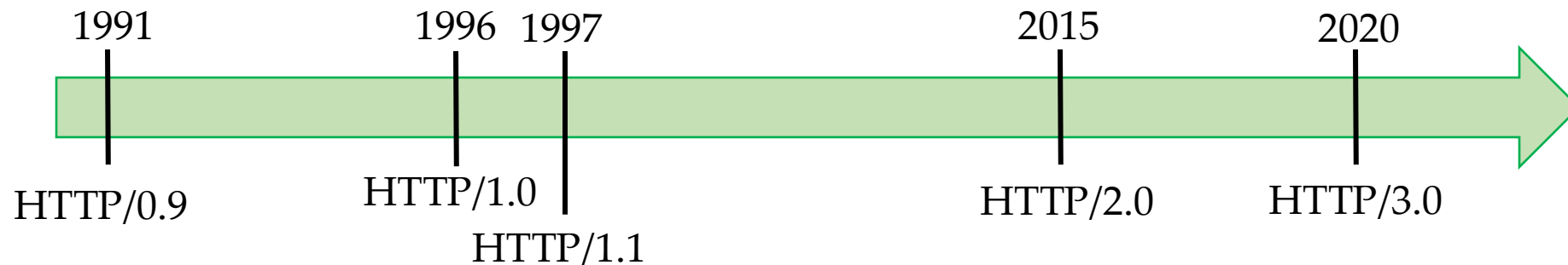
Outline

➤ HyperText Transfer Protocol (HTTP)

➤ Web applications

The HyperText Transfer Protocol (HTTP)

- **HyperText Transfer Protocol (HTTP):** the main web communication protocol for exchanging information between servers and clients.
- Simple **request-response protocol**, proper of the application layer, running over TCP.
- It specifies what messages clients may send to servers and what responses they get back in return.
- Most of Web currently use HTTP/1.1 and HTTP/2.0, which share the same syntax.



Request structure (HTTP/1.1)

- Each **request** consists of one or more lines of ASCII text

Case-sensitive name of the method
(type of the request)

- First line:
METHOD resource protocol
Name of the resource in the server
Name of the protocol version (e.g. HTTP/1.1)

- Request headers:

Host: servername

Additional information related to the request.
In HTTP/1.1 there must be at least one header
containing the DNS name of the server.

Other optional headers

- Empty line
- Request body (payload):

Optional message body

HTTP methods

- HTTP methods indicate the **desired action** to be performed for a given resource

Method	Desired action
GET	Retrieves a resource
HEAD	Retrieves a resource header
POST	Submits an entity to the specified resource (causes a change of server state)
PUT	Replaces all current representations of the target resource with the request payload.
DELETE	Deletes a resource
TRACE	Echoes the incoming request
CONNECT	Establishes a tunnel to the server identified by the target resource
OPTIONS	Describes the communication option for a resource
PATCH	Applies partial modifications to a resource

Examples of request headers

- *Host*: the server's DNS name
- *Authorization*: needed for pages that are protected. The client might need to demonstrate it has the right to see the requested page.
- *Content-Type*: the type of the content in the request body.

(Other) Examples of request headers

BONUS

- *User-Agent*: it allows the client to inform the server about the user agent making the request, e.g., the browser implementation
- *Accept*: it allows to tell the server what the client is willing to accept
- *If-Modified-Since*: time and date of last page modification, it allows the client to check the validity of cached pages (the client stores recently requested contents in a cache memory, similarly to what the server does).
- *Cache-Control*: it specifies directives that must be obeyed by all caching mechanisms along the request/response chain (e.g., *no-cache*).
- ...

Response structure (HTTP/1.1)

- A response message is sent by a server to a client as a reply to its former request message.

Three-digit integer code representing the result of the server's attempt to understand and satisfy the request.

- Status line:

Protocol status_code reason_phrase

Optional. If the status code indicated a problem, the *reason phrase* might displayed the further information about the nature of the problem.

- Response headers:

Optional headers

- Empty line

- Request body (payload):

Optional message body

Status code	Description
1xx	Informational: the request was received and is being processed,
2xx	Successful: the request was successfully received, understood, and accepted.
3xx	Redirecting: further action needs to be taken in order to complete the request.
4xx	Client error: the request contains bad syntax or cannot be fulfilled.
5xx	Server error: The server failed to fulfill an apparently valid request.

Examples of response headers

- *Date*: date and time the message was sent
- *Content-Type*: the type of the returned content (e.g., HTML, image, plain text, ...)
- *Content-Length*: the content length in bytes
- *Last-Modified*: time and date the page was last changed
- ...

(Other) Examples of response headers

BONUS

- *Content-Language*: the natural language used in the page
- *Expires*: Time and date when the page stops being valid
- *ETag*: tag for the content of the page (used for caching)
- *Location*: redirect the recipient to a location other than the requested URL for completion of the request or identification of a new resource
- ...

GET

➤ **GET:** Retrieves a resource.

GET / HTTP/1.1

Host: www.example.com

Accept-Language: en

Request example:

```
GET / HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Response example:

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 155
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close

<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML document.</p>
  </body>
</html>
```

POST

- **POST:** Submits an entity to the specified resource. Can be used for creating a resource.
- If the request is accepted, then the server does something with the data that depends on the URL, and send a response page indicating the result.
 - POST is commonly used when forms are submitted, or when uploading files to servers, e.g., to extend a database appending some new data.

POST HTML Form Example

```
POST /echo/post/form HTTP/1.1
Host: reqbin.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 23

key1=value1&key2=value2
```

POST JSON Data Example

```
POST /echo/post/json HTTP/1.1
Host: reqbin.com
Accept: application/json
Content-Type: application/json
Content-Length: 81

{
  "Id": 78912,
  "Customer": "Jason Sweet",
  "Quantity": 1,
  "Price": 18.00
}
```

Other important methods

- **PUT:** Replaces all current representations of the target resource with the request payload. Can be used for updating/modifying a resource.
- **DELETE:** Deletes a resource.
- **PATCH:** Applies partial modifications to a resource.
- **HEAD:** it just asks for the message header, without the actual content. It can be used to collect information for indexing purposes, or to test a resource for validity.

POST vs PUT vs PATCH

➤ Difference between POST and PUT?

- **POST**: the client asks to the server to accept the entity enclosed in the request as a new subordinate of the resource identified by the request URL.
- **PUT**: the client asks to the server to store the enclosed entity under the request URL. → If the request URL refers to an already existing resource, the server replaces the existing entity with the new enclosed entity. If the URL does not point to an existing resource, the server can create the resource with that URL.

➤ Difference between PUT and PATCH?

- **PUT** supplies a modified version of the resource to be allocated at the request URL.
- **PATCH** supplies a set of instructions to modify the resource at the requested URL.

Other (less) important methods

BONUS

- **TRACE:** it asks the server to send back the request. Useful when requests are not processed correctly to check what the server received.
- **CONNECT:** it lets a user make a connection to a Web server through an intermediate device, such as a Web cache.
- **OPTIONS:** it queries the server for a resource and obtain the methods and headers that can be used with that resource.

Properties of the HTTP methods

- **Safe methods:** A request method is safe if a request with that method has no intended effect on the server.
 - GET, HEAD, OPTIONS, and TRACE are safe, since they only read data from the server.
 - POST, PUT, DELETE, PATCH, CONNECT are not safe, since they may modify the server state.
- **Idempotent methods:** A request method is idempotent if multiple identical requests with that method have the same intended effect as a single such request.
 - PUT and DELETE, and safe methods are idempotent.
 - POST, PATCH and CONNECT are not necessarily idempotent. Sending an identical POST request multiple times may further modify the state of the server.
- **Cacheable methods:** A request method is cacheable if responses to requests with that method may be stored by the client in the cache for future reuse.
 - Only GET, HEAD, and POST are cacheable.

Summary of properties of the HTTP methods

METHOD	REQUEST HAS BODY	RESPONSE HAS BODY	IS SAFE	IS IDEMPOTENT	IS CACHEABLE
GET	Optional	Yes	Yes	Yes	Yes
HEAD	Optional	No	Yes	Yes	Yes
POST	Yes	Yes	No	No	Yes
PUT	Yes	Yes	No	Yes	No
DELETE	Optional	Yes	No	Yes	No
TRACE	No	Yes	Yes	Yes	No
CONNECT	Optional	Yes	No	No	No
OPTIONS	Optional	Yes	Yes	Yes	No
PATCH	Yes	Yes	No	No	No

Example: inspecting the HTTP requests in Chrome

The screenshot shows a web browser with the address bar at `gcappon.github.io/teaching/`. The website displays a profile for Giacomo Cappon, an Assistant Professor at DEI, University of Padova, YOLO committer, located in Padova, Italy. His research interests include Biomedical Wearable Technologies for Healthcare and Wellbeing. The page lists two Master's courses: "Biomedical Wearable Technologies for Healthcare and Wellbeing, 2023/24" and "Biomedical Wearable Technologies for Healthcare and Wellbeing, 2022/23".

The Chrome DevTools Network tab is open, showing a list of requests. The "teaching/" request is selected, and its details are visible in the right pane. The "Request headers" box points to the "teaching/" request in the list. The "Requested page and contents" box points to the "teaching/" request in the list. The "Response headers" box points to the "Response Headers" section in the details pane.

Request headers

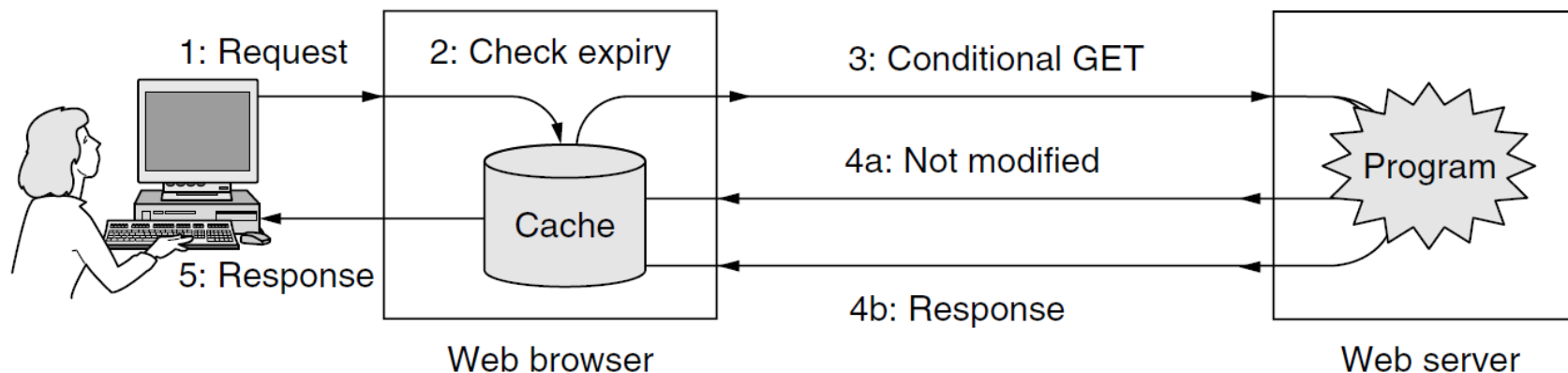
Requested page and contents

Response headers

Name	Value
Request URL:	https://gcappon.github.io/teaching/
Request Method:	GET
Status Code:	200 OK
Remote Address:	185.199.110.153:443
Referrer Policy:	strict-origin-when-cross-origin
Response Headers	
Accept-Ranges:	bytes
Access-Control-Allow-Origin:	*
Age:	0
Cache-Control:	max-age=600
Content-Encoding:	gzip
Content-Length:	2891
Content-Type:	text/html; charset=utf-8
Date:	Thu, 01 Feb 2024 12:49:10 GMT
Etag:	W/"64d79e9b-293c"
Expires:	Thu, 01 Feb 2024 12:59:10 GMT

Caching

- People often return to Web pages that they have viewed before, and related Web pages often have the same embedded resources.
- **Caching:** save on the client pages that are fetched for possible subsequent use.
- HTTP headers can be used to let the client know when a page can be safely re-used. → reduced network traffic
 - **Page validation:** The cache is consulted, and if it has a copy of a page for the requested URL that is known to be fresh (i.e., still valid), there is no need to fetch it new from the server. The validity time is obtained by the *Expires* header of the original GET request.
 - **Conditional GET:** The client checks the time a cached page was last updated from the *Last-Modified* header. It can send it to the server using the *If-Modified-Since* header to ask for the page only if it has been changed in the meantime.



Web communication protocols

- HTTP is not the only Web communication protocol.
- Different protocols have been defined for different purposes.

Name	Used for	Example
http	Hypertext (HTML)	http://www.ee.uwa.edu/~rob/
https	Hypertext with security	https://www.bank.com/accounts/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Local file	file:///usr/suzanne/prog.c
mailto	Sending email	mailto:JohnUser@acm.org
rtsp	Streaming media	rtsp://youtube.com/montypython.mpg
sip	Multimedia calls	sip:eve@adversary.com
about	Browser information	about:plugins

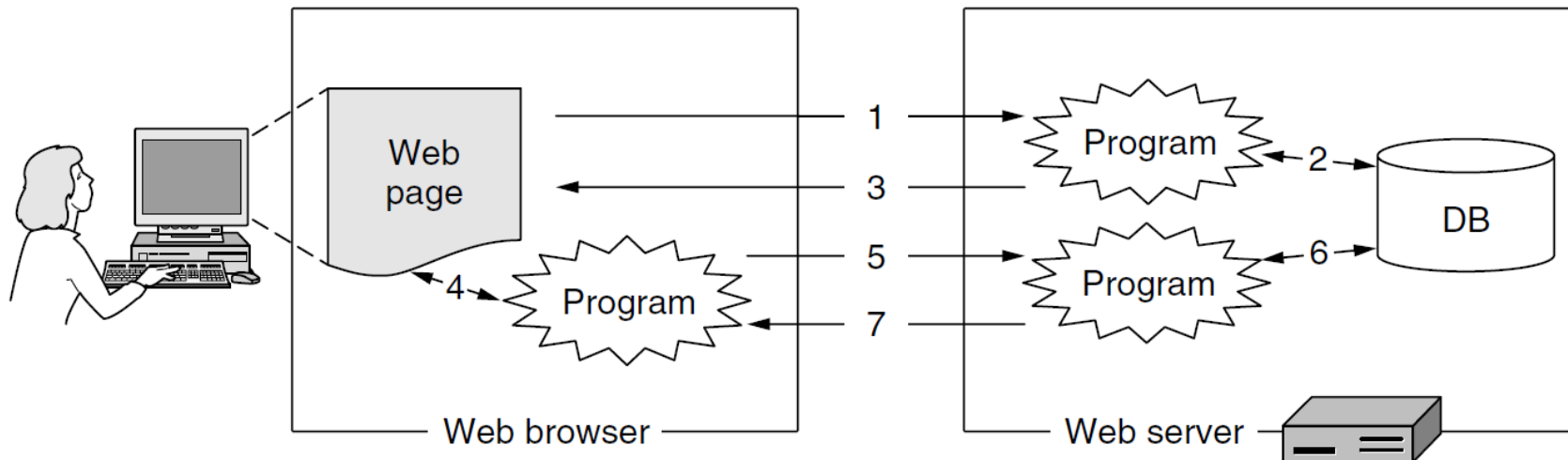
Outline

➤ HyperText Transfer Protocol (HTTP)

➤ Web applications

Web application

- **Web application:** a Web page or a collection of Web pages, delivered over HTTP, which use server-side or client-side processing, i.e., programs, to provide an "application-like" experience within a Web browser.
- Web applications access data that are stored in databases in the server and use Web protocols (e.g., HTTP) to exchange data with the client. They use the Web browser to display a user interface by which the user can use the app (making queries, inserting data, view retrieved data, etc.).
- Advantage of Web applications: The users do not need to install separate application programs, and data can be accessed from different computers and backed up by the service operator.
- Examples of Web apps: Gmail, Google Docs, Trello, online banking apps, online retail sales apps, ...



To act as applications, Web pages can no longer be static, but they need to generate **dynamic content**.

Server-side dynamic Web page generation

- A simple situation in which server-side processing is needed is a **Web form**.
- When the user compiles the form and click 'Submit order', the client sends a **POST request** to a specific URL pointing to a **program** in the Web server.
- The POST request will include the input data of the form, properly formatted, in the request body.
- The server receives the data and passes them to the program.
- The program process the data and generates an HTML page, whose content depends on the inserted data.
- The server returns the HTML page to the client's browser, in the body of the POST response.

Widget Order Form

Name

Street address

City State Country

Credit card # Expires M/C ☐ Visa ☐

Widget size Big ☐ Little ☐ Ship by express courier ☐

Thank you for ordering an AWI widget, the best widget money can buy!

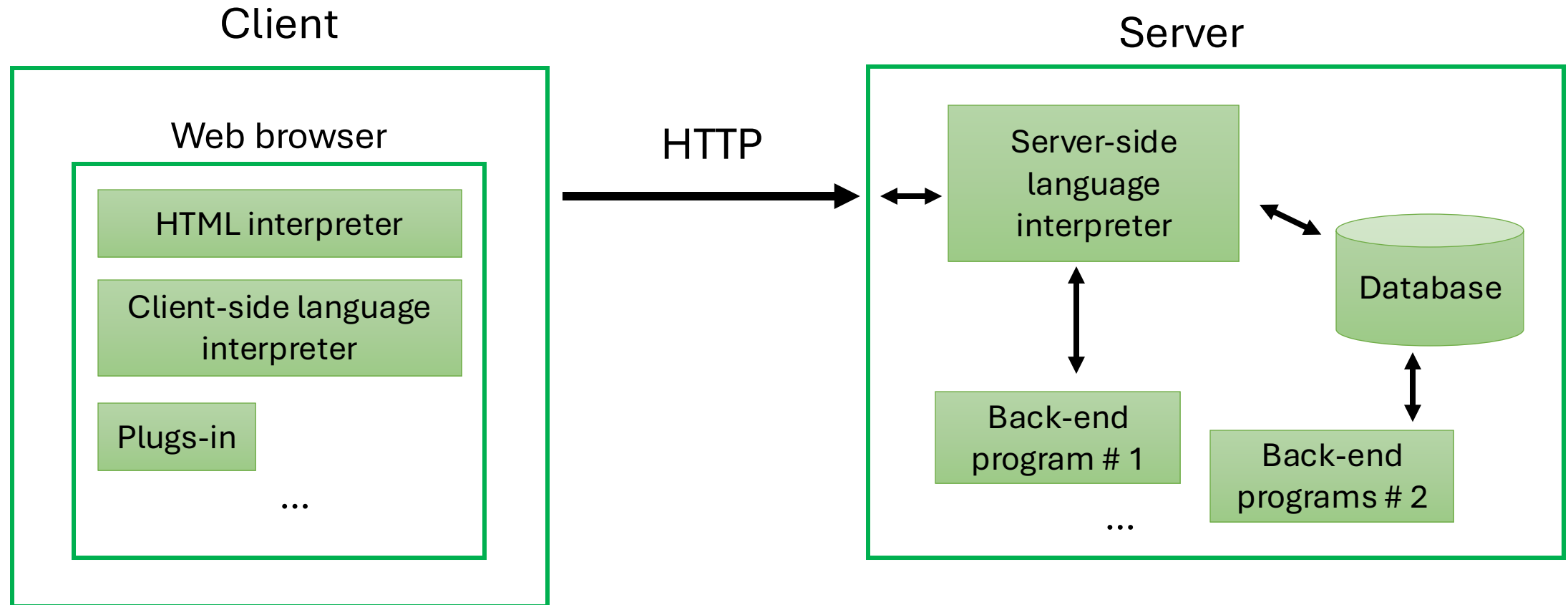
How can the server invoke the program?
Using specific APIs (Application Programming Interfaces)

Client-side dynamic Web page generation

- Client-side processing is necessary to have applications with a more responsive interface that quickly responds to user commands and interact with the user.
- It is necessary to have scripts embedded in HTML pages that are executed on the client machine.
- The most popular language for client-side scripting is **JavaScript**.
- When the browser interprets the HTML page, it also interprets the JavaScript code contained in the page.
- All the work is done locally, inside the browser. There is no contact with the server.

```
<html>
<head>
<script language="javascript" type="text/javascript">
function response(test form) {
var person = test form.name.value;
var years = eval(test form.age.value) + 1;
document.open();
document.writeln("<html> <body>");
document.writeln("Hello " + person + ".<br>");
document.writeln("Prediction: next year you will be " + years + ".");
document.writeln("</body> </html>");
document.close(); }
</script>
</head>
<body>
<form>
Please enter your name: <input type="text" name="name">
<p>Please enter your age: <input type="text" name="age"><p>
<input type="button" value="submit"
onclick="response(this.form)">
</form>
</body>
</html>
```

Example of a Web application architecture



Structured data formats

- Web applications need to exchange data with the server.
- HTML is not suitable for representing data because it mixes content with formatting. It is indeed concerned about the presentation of the content.
 - It is difficult to automatically retrieve data within an HTML file
- Two popular structured data formats that facilitate the automatic structured data processing:
 - Extensible Markup Language (XML)
 - JavaScript Object Notation (JSON)

Extensible Markup Language (XML)

- A language for specifying structured content.
- Introduced in 1998.
- It defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.
- Unlike HTML, there are no defined tags for XML. Each user can define their own tags.
- Extension .xml

Example of XML code that define a book list

```
<?xml version="1.0" ?>
<book list>
<book>
<title> Human Behavior and the Principle of Least Effort </title>
<author> George Zipf </author>
<year> 1949 </year>
</book>
<book>
<title> The Mathematical Theory of Communication </title>
<author> Claude E. Shannon </author>
<author> Warren Weaver </author>
<year> 1949 </year>
</book>
<book>
<title> Nineteen Eighty-Four </title>
<author> George Orwell </author>
<year> 1949 </year>
</book>
</book list>
```

JavaScript Object Notation (JSON)

- A format that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and arrays.
- Introduced in 2001.
- JSON is language-independent. It was derived from JavaScript, but many modern programming languages include code to generate and parse JSON-format data.
- Extension .json
- JSON data types: number, string, Boolean, null (empty value)
- JSON data structures
 - Object
 - Array

JSON data structures

- **JSON object:** a collection of name–value pairs where the names (also called keys) are strings. The order of pairs is not relevant. The object is surrounded by curly braces.

```
{
  "employee": {
    "id": 1,
    "name": "Admin",
    "location": "USA"
  }
}
```

Objects can contain array and arrays can contain objects.

- **JSON array:** an ordered list of zero or more elements, each of which may be of any type. Arrays use square bracket notation with comma-separated elements.

```
{
  "employees": [
    {
      "id": 1,
      "name": "Admin",
      "location": "India"
    },
    {
      "id": 2,
      "name": "Author",
      "location": "USA"
    },
    {
      "id": 3,
      "name": "Visitor",
      "location": "USA"
    }
  ]
}
```

Why JSON are important here? What we need?

- The Fitbit data you will need to develop your project (stored in the IMPACT server are stored in a JSON format
- We need to learn how to make and "debug" our requests!



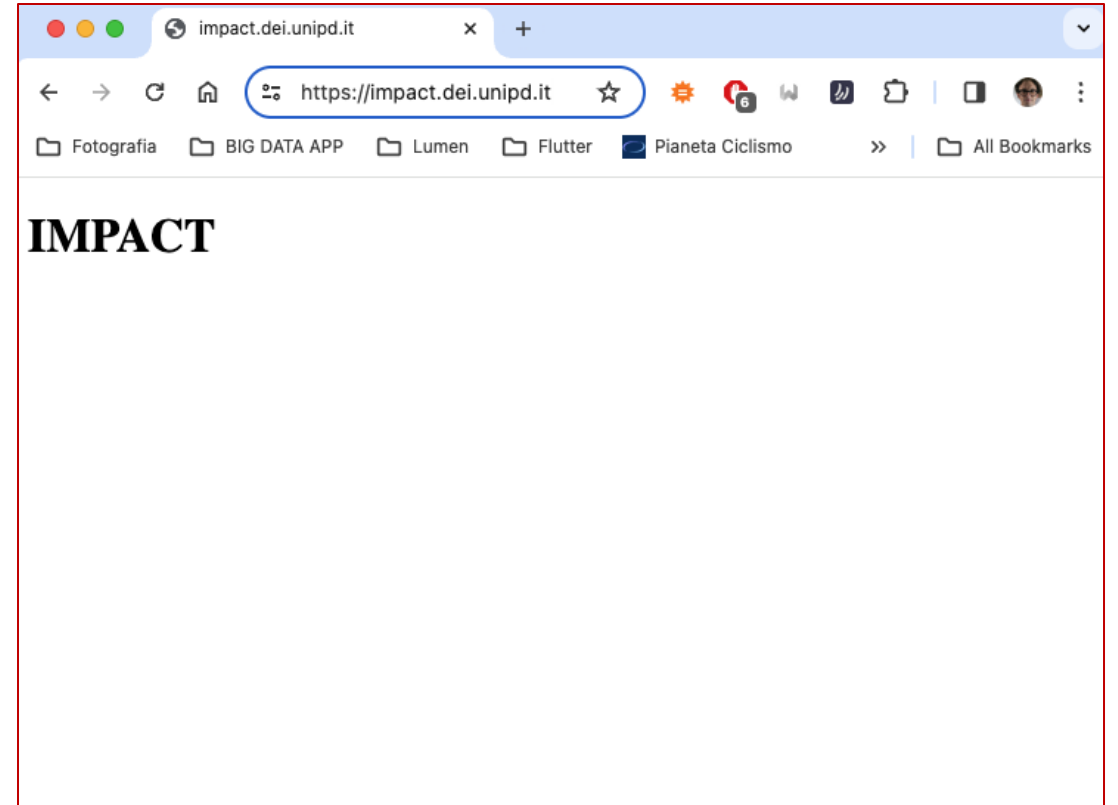
POSTMAN

- We saw that using a browser we are actually visualizing the response of an HTTP request using the GET method.
- What if we need to make an authenticated request?
What if we need to make a POST request?
- POSTMAN can help us here:
<https://www.postman.com>
- We will use it during the theory lessons to try what we learn



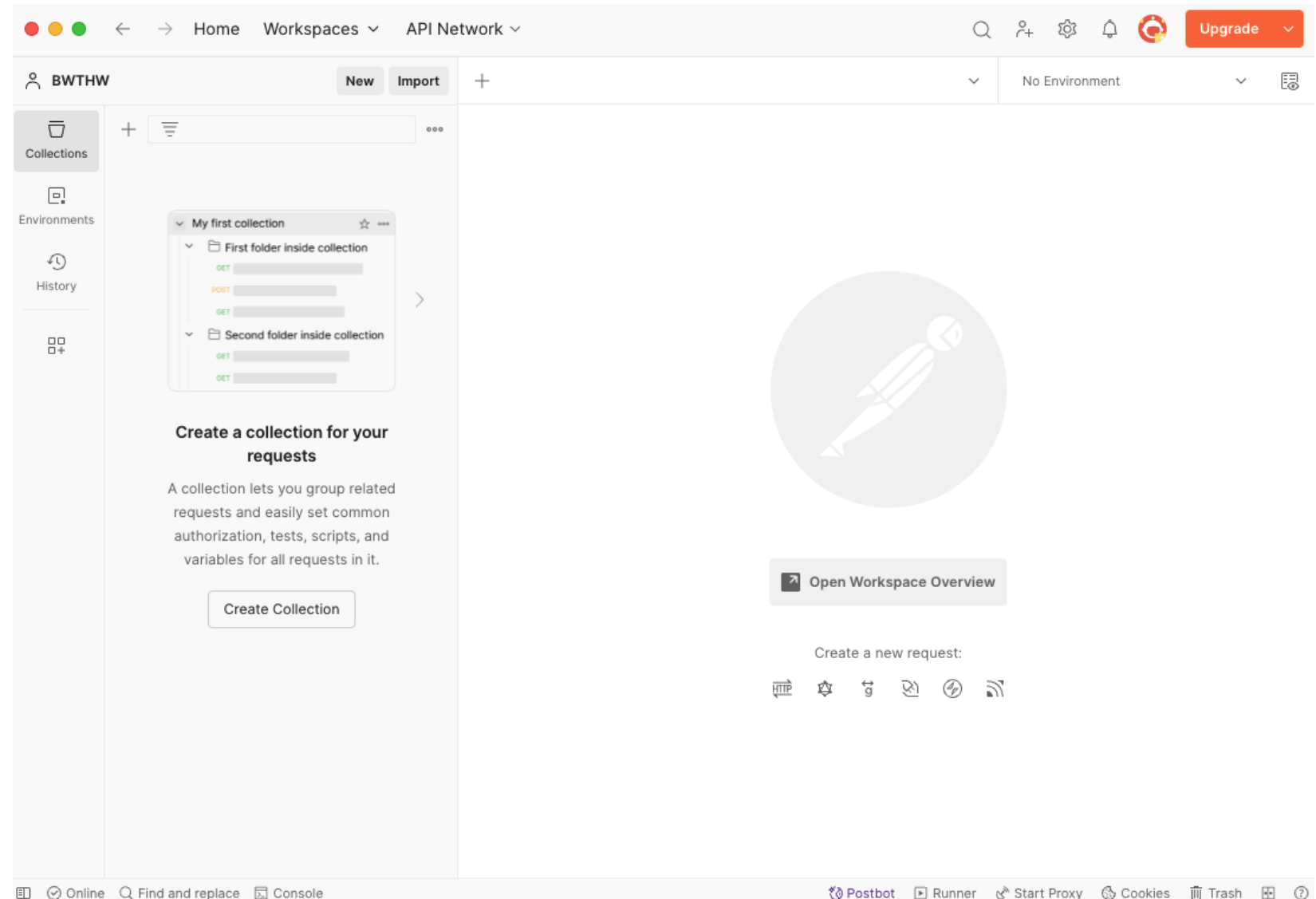
Hello IMPACT

- At <https://impact.dei.unipd.it/> you will obtain the following
- Let's replicate it in POSTMAN



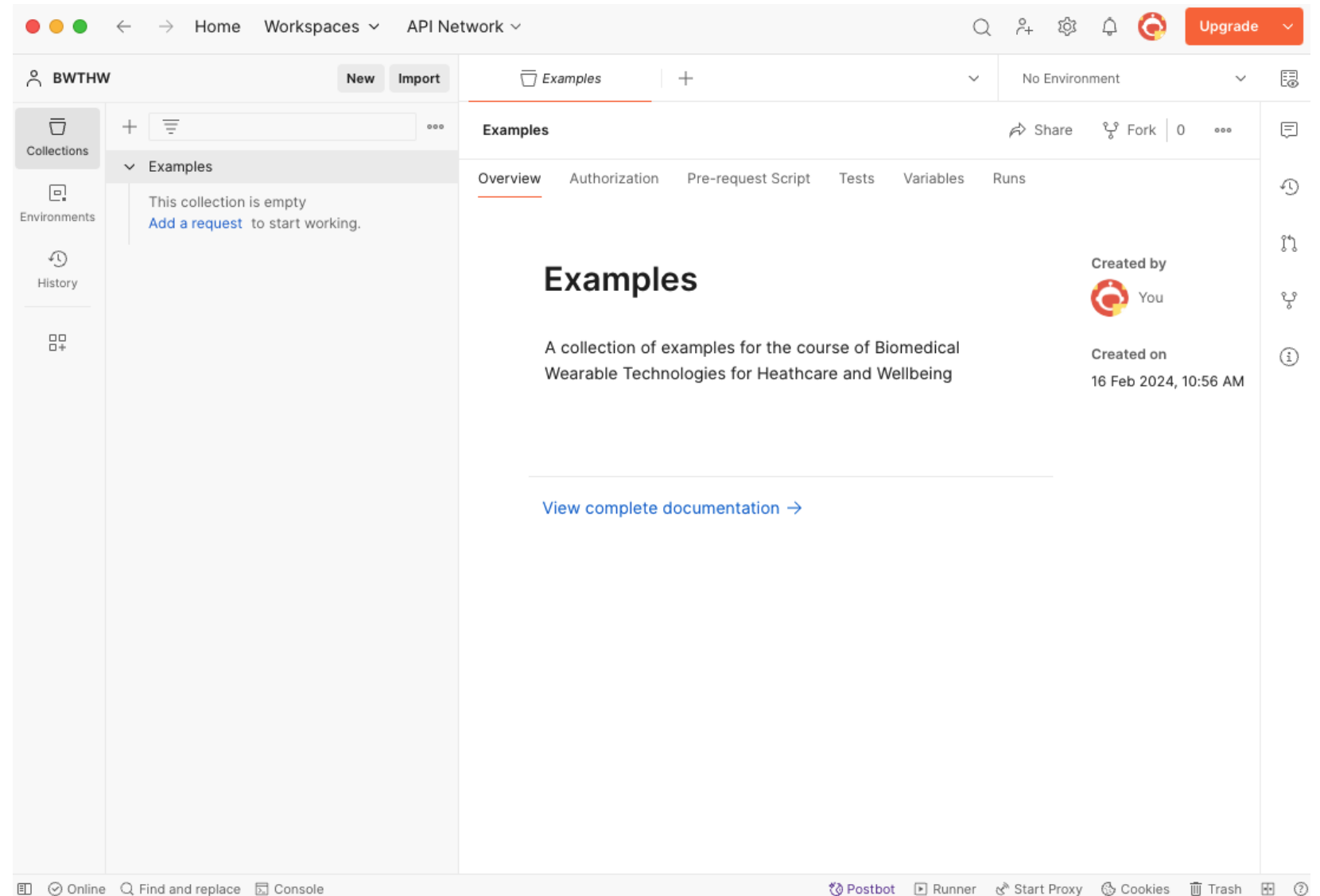
Hello IMPACT

- This is the POSTMAN interface
- Many things can be done (e.g., automatic debugging, writing custom scripts, etc...)
- We will limit to create and organize a collection of HTTP requests



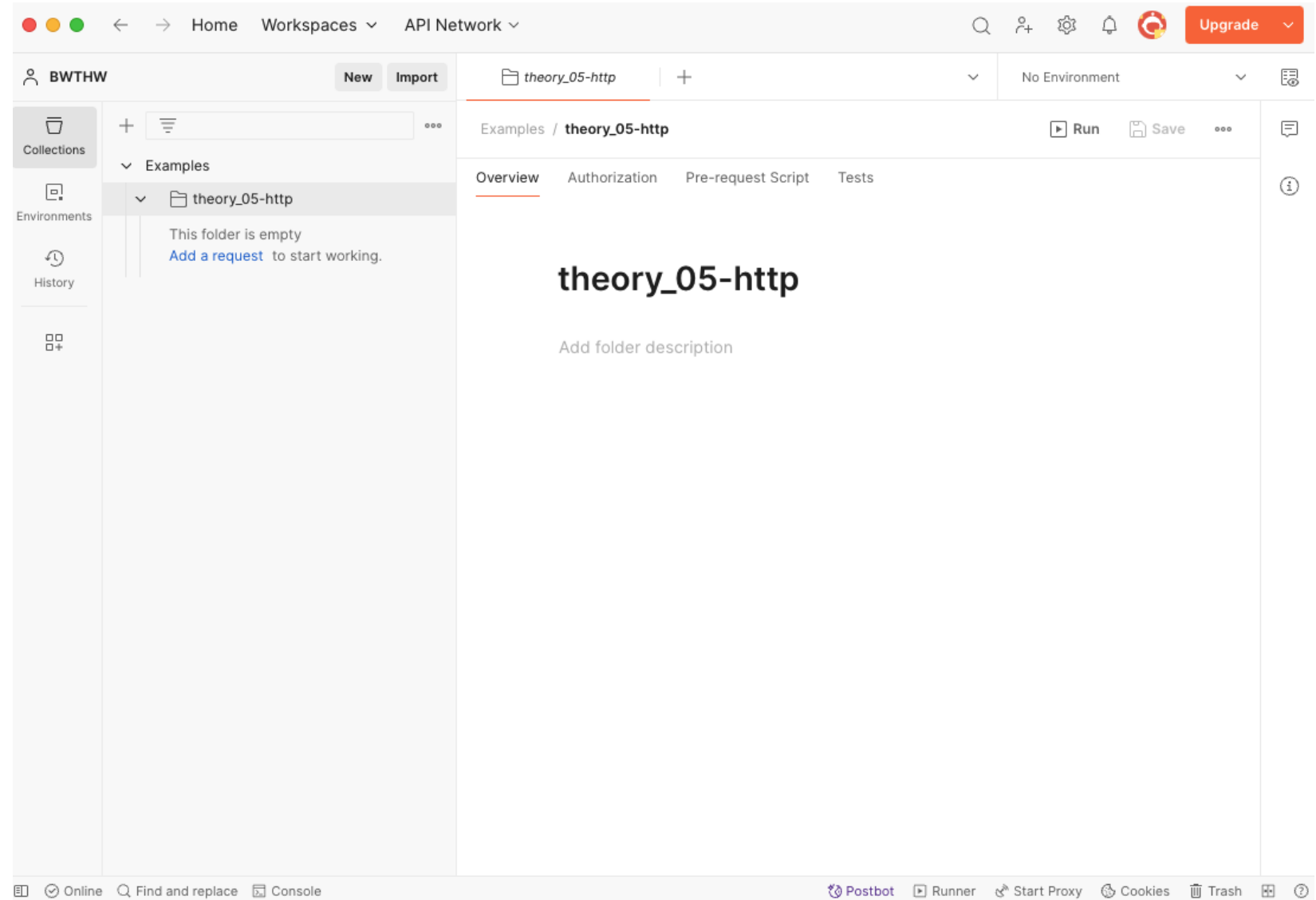
Hello IMPACT

- Go to the "Collections" tab
- Press the "+" button
- Create a "Blank collection"
- Call it "Examples": here we will collect all the theory lessons examples



Hello IMPACT

- Right click the “Examples” collection
- Select “Add folder”
- Call it as you want, e.g.: “theory_05-http”



Hello IMPACT

- Right click the folder and choose “Add request”
- Call it as you want, e.g., “Hello IMPACT”
- Choose the GET method
- And write the IMPACT home URL, i.e., <https://impact.dei.unipd.it/>
- Click “Send” and inspect the response

The screenshot shows the Postman API client interface. On the left, the 'Collections' sidebar shows a folder named 'theory_05-http' with a sub-collection 'Hello IMPACT'. The main panel displays the details of the 'GET Hello IMPACT' request. The URL is 'https://impact.dei.unipd.it'. The 'Send' button is visible. Below the URL bar, the 'Query Params' table is empty. The 'Body' tab is selected, showing the response in 'Pretty' format. The response is an HTML document with a single heading 'IMPACT'.

Key	Value	Description
Key	Value	Description

```
1 <!DOCTYPE html>
2 <html>
3
4 <body>
5
6   <h1>IMPACT</h1>
7
8 </body>
9
10 </html>
```

References

- Tanenbaum, Wetherall – Computer Networks – Fifth Edition
 - Chapter 7 – The application layer
- POSTMAN: <https://www.postman.com>